

CS6910: Fundamentals of Deep Learning

Programming Assignment II

Submitted by Team 23

Gauni Kaushik - CS25M021

Vadithe Sai Kumar Naik - CS25M053

1 Task 1: Image Classification using Deep CNN

1.1 Task 1(a): VGG16 Feature Extraction

- **Model:** VGG16 (pretrained on ImageNet) used as a fixed feature extractor.
- **Feature Extraction:** Features were obtained from the final max-pooling layer and flattened into a 4096-dimensional vector.
- **Classifier:** A Multi-Layer Feed Forward Neural Network (MLFFNN) with:
 - Two hidden layers: 1024 and 256 neurons
 - Activation: ReLU
 - Regularization: Dropout (0.5)
- **Training Details:**
 - Optimizer: Adam
 - Learning Rate: 0.0005
 - Batch Size: 32
 - Number of Epochs: 20

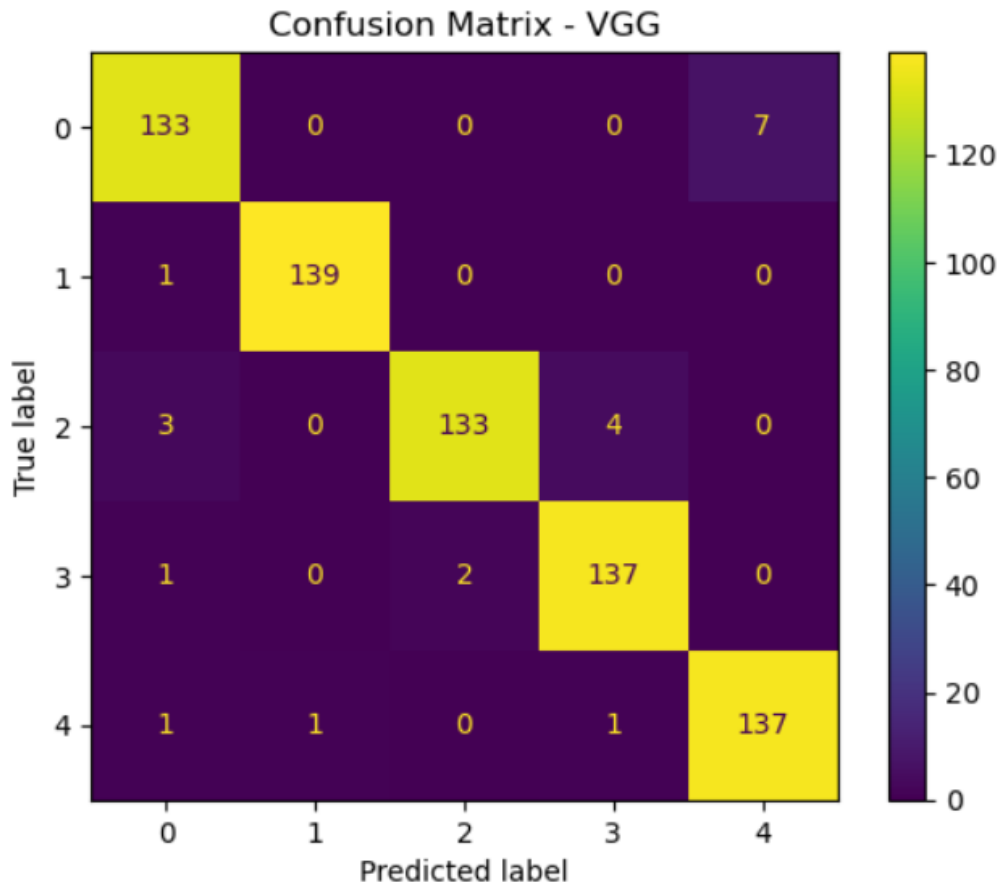
Training Progress:

Epoch	Loss	Accuracy
1	22.8927	0.9671
2	2.8414	0.9629
3	0.3710	0.9629
4	0.1068	0.9729
5	0.0132	0.9686
6	0.0053	0.9671
7	0.0029	0.9771
8	0.0019	0.9771
9	0.0013	0.9757
10	0.0010	0.9871

Final Results:

Feature Extractor	Test Accuracy	Final Test Loss
VGG16	98.29%	0.001

Confusion Matrix:



Observations:

- The model learns quickly as the loss decreases fast and accuracy reaches about 98%.
- Accuracy becomes stable after a few epochs, showing the model has converged well.
- The confusion matrix has most values on the diagonal, meaning most predictions are correct.
- Only a few small errors occur between similar classes, so overall performance is very good.

1.2 Task 1(b): GoogLeNet Feature Extraction

- **Model:** GoogLeNet pretrained on ImageNet, used as a feature extractor.
- **Feature Extraction:** Features were obtained from the Global Average Pooling layer, resulting in a 1024-dimensional feature vector.
- **Classifier:** A Multi-Layer Feed Forward Neural Network (MLFFNN) with:
 - Two hidden layers: 512 and 128 neurons
 - Activation: ReLU
 - Regularization: Dropout (0.5)
- **Training Details:**
 - Optimizer: Stochastic Gradient Descent (SGD) with momentum
 - Learning Rate: 0.001
 - Batch Size: 32
 - Number of Epochs: 10

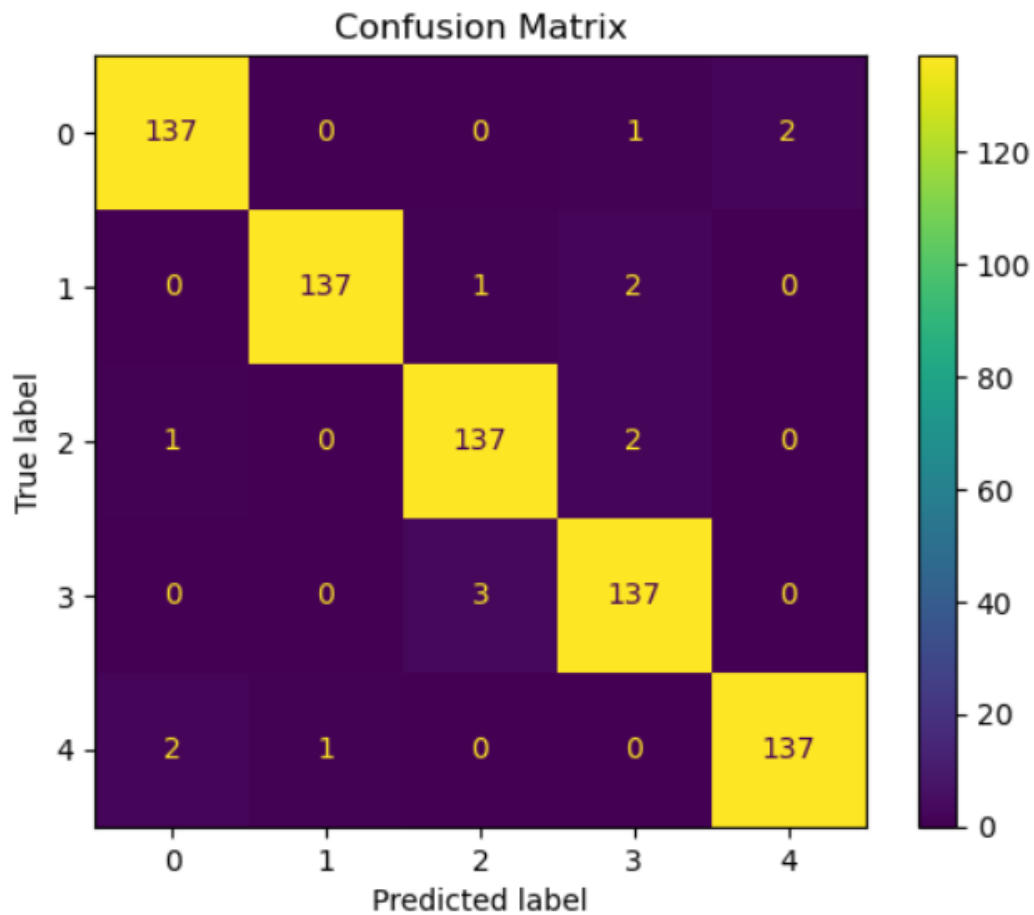
Training Progress:

Epoch	Loss	Accuracy
1	113.3558	0.9614
2	27.0733	0.9643
3	15.4665	0.9714
4	11.8400	0.9771
5	10.2882	0.9800
6	9.6422	0.9814
7	7.9111	0.9757
8	6.8329	0.9771
9	6.6105	0.9757
10	5.8013	0.9786

Final Results:

Feature Extractor	Test Accuracy	Final Test Loss
GoogLeNet	97.85%	5.80

Confusion Matrix:



Observations:

- The model shows steady improvement as loss decreases and accuracy increases up to about 98%.
- Accuracy is consistently high after a few epochs, indicating good learning and stable performance.
- The confusion matrix has strong diagonal values, showing most predictions are correct.
- Only a few small mistakes occur between nearby classes, so overall classification performance is very good.

1.3 Comparative Results

Feature Extractor	Test Accuracy	Final Test Loss
VGG16	98.29%	0.0241
GoogLeNet	97.85%	0.0512

Observations:

- VGG16 gives slightly better accuracy and lower loss compared to GoogLeNet.
- VGG16 uses more features, so it can learn more details but takes more computation.
- GoogLeNet uses fewer features, so it is faster and more efficient.
- There is a trade-off: VGG16 gives better performance, while GoogLeNet is more lightweight.

2 Task 2: Custom CNN

2.1 Model Configuration:

- **Model:** Custom shallow Convolutional Neural Network (CNN) trained from scratch.
- **Input:** RGB images of size $64 \times 64 \times 3$.
- **Architecture:**
 - **Layer 1 (Conv1):** Conv2d ($3 \rightarrow 4$ filters), 3×3 kernel, stride 1, padding 1
Output size: $64 \times 64 \times 4$
 - **Activation:** ReLU
 - **Layer 2 (Pool1):** Mean Pooling (2×2 , stride 2)
Output size: $32 \times 32 \times 4$
 - **Layer 3 (Conv2):** Conv2d ($4 \rightarrow 8$ filters), 3×3 kernel, stride 1, padding 1
Output size: $32 \times 32 \times 8$
 - **Activation:** ReLU
 - **Layer 4 (Pool2):** Mean Pooling (2×2 , stride 2)
Output size: $16 \times 16 \times 8$
 - **Flatten Layer:** Converts feature map to vector of size $8 \times 16 \times 16 = 2048$
 - **Fully Connected Layer:** Linear ($2048 \rightarrow$ number of classes)
- **Classifier:** Single fully connected layer for final prediction.

- **Training Details:**

- Loss Function: Cross-Entropy Loss
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 32
- Number of Epochs: 10

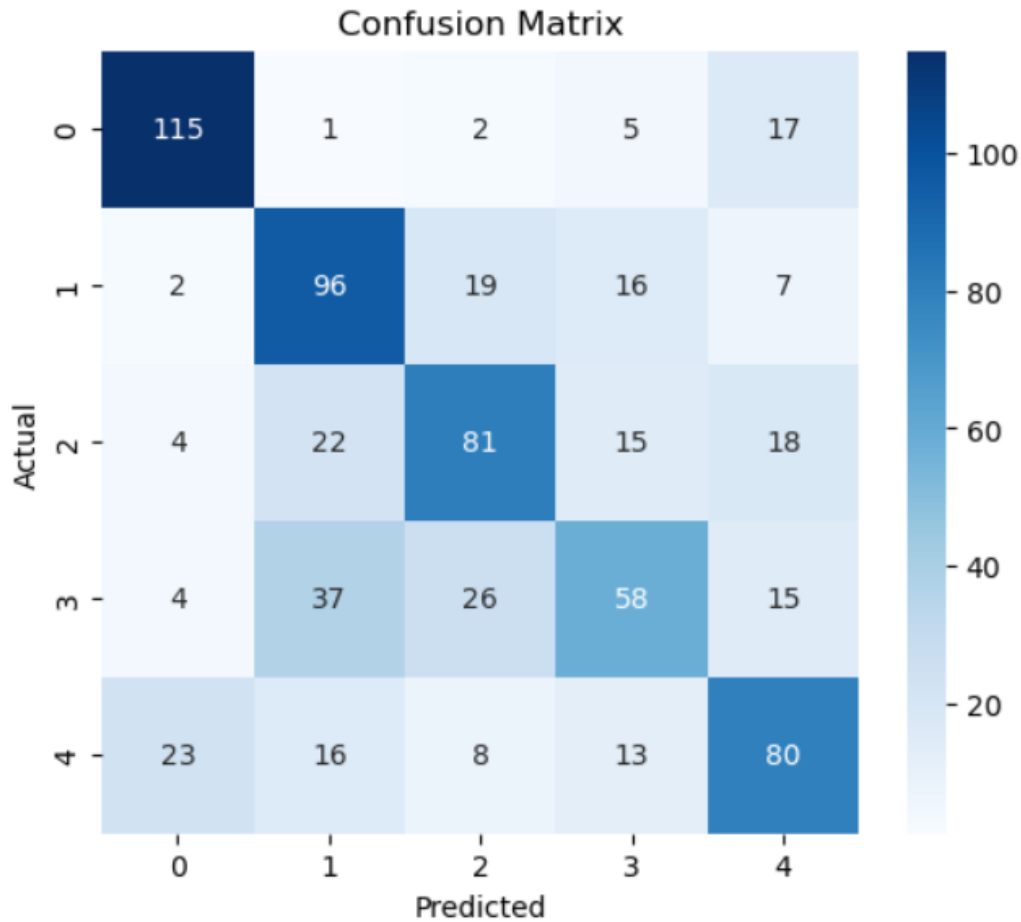
2.2 Training Result:

Epoch	Loss	Accuracy
1	139.1601	0.4680
2	114.8134	0.5969
3	109.1780	0.6246
4	105.9930	0.6283
5	104.0657	0.6397
6	100.0726	0.6591
7	98.5974	0.6623
8	95.3279	0.6794
9	94.8554	0.6777
10	92.8372	0.6851

Final Results:

Feature Extractor	Test Accuracy	Final Test Loss
Custom CNN	61.43%	92.83

Confusion Matrix:



2.3 Observations:

The model achieved a **Test Accuracy of 61.14%**.

- The model gives low accuracy (around 61%), showing it is not learning well.
- The confusion matrix has many off-diagonal values, meaning many wrong predictions.
- Some classes are highly confused with each other, especially class 1, 2, and 3.
- Only a few correct predictions appear on the diagonal, so overall performance is weak.

Overall, the performance gap compared to transfer learning demonstrates the importance of depth and prior knowledge in visual tasks.

3 Task 3: Sentiment Analysis using RNN

3.1 Model Configuration:

- **Model:** Many-to-one Recurrent Neural Network (RNN) for binary sentiment classification.
- **Input Representation:**
 - Text converted to sequences using a vocabulary
 - Pretrained GloVe embeddings (200 dimensions)
 - Maximum sequence length: 100 tokens (padded/truncated)
- **Architecture:**
 - **Embedding Layer:** Converts tokens to 200-dimensional vectors (frozen embeddings)
 - **RNN Layer:** Vanilla RNN with hidden size = 25
 - **Dropout Layer:** Dropout with probability 0.2 for regularization
 - **Fully Connected Layer:** Linear ($25 \rightarrow 2$ classes)
- **Output:** Final hidden state is used for binary classification (many-to-one setup).
- **Training Details:**
 - Loss Function: Weighted Cross-Entropy Loss
 - Class Weights: 1.0 (Negative), 2.5 (Positive)
 - Optimizer: Adam
 - Learning Rate: 0.0005
 - Batch Size: 64
 - Number of Epochs: 30
- **Challenge:** Dataset is highly imbalanced (93% negative samples).
- Class-weighted loss was used to give more importance to the minority (positive) class.

3.2 Training Results

Epoch	Loss	Accuracy
20	0.1693	0.9607
21	0.1533	0.9652
22	0.1486	0.9648
23	0.1405	0.9675
24	0.1344	0.9695
25	0.1281	0.9736
26	0.1271	0.9712
27	0.1179	0.9748
28	0.1133	0.9743
29	0.1093	0.9759
30	0.1031	0.9775

Final Result:

	Predicted Neg	Predicted Pos
Actual Neg	1281	13
Actual Pos	61	45

Negative Sentiment:

Correct Predictions: 1281

Wrong Predictions: 13

Accuracy: 99.00%

Positive Sentiment:

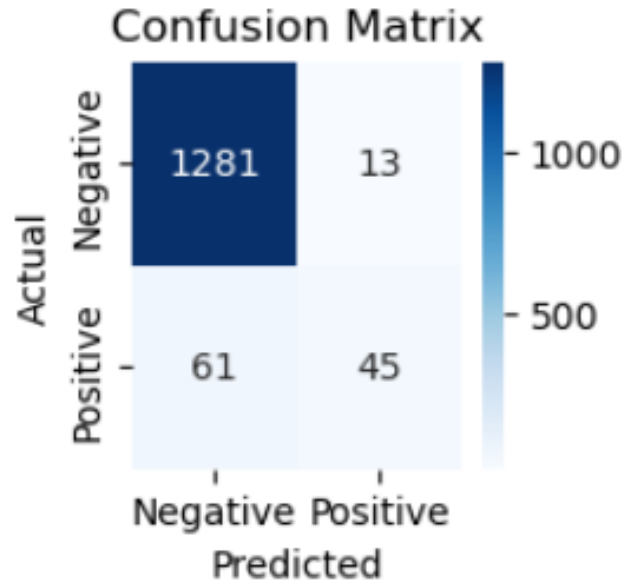
Correct Predictions: 45

Wrong Predictions: 61

Accuracy: 42.45%

Overall Model Accuracy: 94.71%

Confusion Matrix:



Observations:

- The model achieves high overall accuracy (around 94.7%) and learns well as loss decreases steadily.
- The confusion matrix shows very high accuracy for the negative class with most predictions correct.
- The model performs poorly on the positive class, with many positive samples predicted as negative.
- This happens due to class imbalance, so the model is biased toward the majority (negative) class.

4 Task 4: Part-of-Speech Tagging using RNN

4.1 Model Configuration:

- **Model:** Many-to-many Recurrent Neural Network (RNN) for POS tagging.
- **Input Representation:**
 - Sentences tokenized into words
 - Pretrained GloVe embeddings (200 dimensions)
 - Maximum sequence length: 50 tokens (padded/truncated)

- **Architecture:**

- **Embedding Layer:** Converts words to 200-dimensional vectors (frozen embeddings)
- **RNN Layer:** RNN with hidden size = 25
- **Fully Connected Layer:** Linear (25 $\rightarrow$ number of POS tags, $\approx$ 12)

- **Training Details:**

- Loss Function: Cross-Entropy Loss (ignoring padding tokens)
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 32
- Number of Epochs: 10

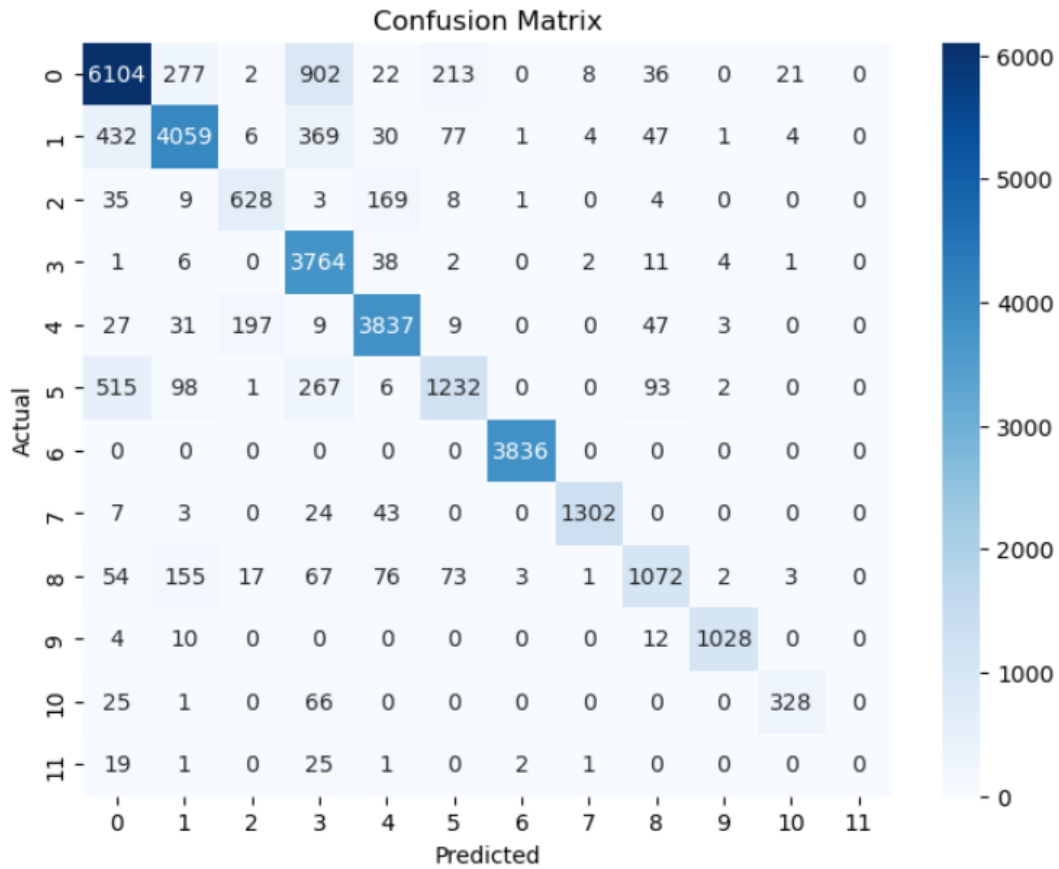
4.2 Training Results:

Epoch	Loss	Accuracy
1	1.5769	0.5678
2	0.8100	0.7549
3	0.6134	0.8088
4	0.5266	0.8333
5	0.4740	0.8498
6	0.4379	0.8607
7	0.4098	0.8697
8	0.3877	0.8759
9	0.3686	0.8815
10	0.3530	0.8866

Final Results:

Feature Extractor	Test Accuracy	Final Test Loss
Custom RNN	85.91%	0.1409

Confusion Matrix:



Observations:

- The model shows steady improvement, with accuracy increasing from about 56% to 88%, indicating good learning over epochs.
- The overall test accuracy (85.91%) is decent, showing the model can capture sequence patterns reasonably well.
- The confusion matrix has strong diagonal values for many classes, meaning most POS tags are predicted correctly.
- Some classes are still confused with each other, especially similar tags, showing limitations of a simple RNN.

5 Conclusion:

- **Task 1(a):** VGG16 achieves high accuracy (98.29%) because pretrained features help the model learn better.
- **Task 1(b):** GoogLeNet performs well (97.85%) and is more efficient, while still giving very high accuracy.
- **Task 2:** Custom CNN gives low accuracy (61.43%) since it learns from scratch and has limited capacity.
- **Task 3:** RNN achieves high accuracy (94.71%) for sentiment analysis but struggles with the minority (positive) class.
- **Task 4:** RNN gives good performance (85.91%) for POS tagging by capturing sequence patterns.